

Raport - Fabryka Czekolady

Projekt Systemów Operacyjnych

Haczyk Kamil

0. Wymagania Systemowe

- System operacyjny: Linux(np.Ubuntu) lub Windows z podsystemem WSL
- Kompilator: gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
- Narzędzie **make** do automatyzacji procesu kompilacji.
- Biblioteki systemowe do obsługi IPC System V (`stdlib.h`, `stdio.h`, `unistd.h`, `sys/types.h`, `sys/ipc.h`, `sys/shm.h`, `sys/sem.h`).

Struktura plików projektu

```
fabryka/
├── include/
│   ├── common.h      # Definicje struktur, stałe, makra
│   └── utils.h        # Deklaracje funkcji pomocniczych
├── src/
│   ├── dyrektor.c     # Proces główny zarządzający fabryką
│   ├── dostawca.c     # Proces dostarczający składniki
│   ├── pracownik.c    # Proces produkujący czekoladę
│   └── utils.c        # Implementacja funkcji pomocniczych
├── bin/               # Pliki wykonywalne (tworzone po kompilacji)
└── Makefile
```

Kompilacja

Projekt wykorzystuje narzędzie `make` do automatyzacji procesu budowania.

Komendy zdefiniowane w Makefile: `make all` (lub po prostu `make`) – Tworzy katalog `bin/` i kompiluje pliki wykonywalne: `dyrektor`, `dostawca`, `pracownik`. *
Flagi kompilatora: `-Wall -Wextra -std=gnu99 -Iinclude -D_GNU_SOURCE` (szczegółowe ostrzeżenia, standard GNU, ścieżka do nagłówków, makra systemowe Linux).

- `make clean` – Usuwa katalog z plikami wykonywalnymi (`bin/`), plik zapisu stanu (`magazyn_stan.dat`) oraz logi (`raport.txt`), przywracając projekt do stanu początkowego.
- `make run` – Kompiluje projekt (jeśli to konieczne) i automatycznie uruchamia proces dyrektora.

Aby wyczyścić projekt ze starych plików, wpisz w terminalu:

```
make clean
```

Aby skompilować projekt, wpisz w terminalu:

```
make
```

Aby uruchomić projekt, wpisz w terminalu:

```
./bin/dyrektor
```

1. Założenia Projektowe

Projekt symuluje fabrykę czekolady wykorzystując mechanizmy komunikacji międzyprocesowej w systemach Unix/Linux:

- **Architektura wieloprotokółowa:** Dyrektor (proces główny) zarządza 4 dostawcami i 2 pracownikami
- **Magazyn:** Implementacja jako pamięć dzielona z osobnymi kolejkami FIFO dla każdego składnika (A, B, C, D)
- **Synchronizacja:** 11 semaforów zapewniających bezpieczeństwo dostępu i koordynację
- **Produkcja:**
 - Czekolada TYP_1: składniki A + B + C (stanowisko 1)
 - Czekolada TYP_2: składniki A + B + D (stanowisko 2)
- **Rozmiary składników:** A=1B, B=1B, C=2B, D=3B
- **Pojemność magazynu:** 50 bajtów (konfigurowalny)

2. Ogólny Opis Kodu

2.1 Komponenty Systemu

Dyrektor:

- Inicjalizuje zasoby IPC (pamięć dzielona, semafony)
- Uruchamia procesy potomne (fork)
- Obsługuje menu interaktywne
- Zarządza cyklem życia fabryki
- Implementuje graceful shutdown

Dostawcy (4 procesy):

- Każdy dostarcza jeden typ składnika (A, B, C lub D)
- Sprawdzają dostępność miejsca w magazynie
- Współpracują przez semafony z pracownikami
- Reagują na sygnały zatrzymania (SIGUSR2, SIGTERM)

Pracownicy (2 procesy):

- Atomowo pobierają komplet składników
- Produkują czekoladę zgodnie ze swoim typem
- Zwalniają miejsce w magazynie po pobraniu
- Reagują na sygnały zatrzymania (SIGUSR1, SIGTERM)

3. Opis Kluczowych Procedur

3.1 Inicjalizacja Magazynu

Magazyn wykorzystuje strukturę **RingQueue** dla każdego składnika, implementując wzorzec **circular buffer (ring buffer)**:

```
typedef struct {
    char dane[KOLEJKA_POJEMNOSC]; // Bufor cykliczny
    int head; // Indeks do wstawiania (producent)
    int tail; // Indeks do pobierania (konsument)
    int count; // Liczba elementów w kolejce
} RingQueue;

typedef struct {
    RingQueue kolejka_A, kolejka_B, kolejka_C, kolejka_D;
    int suma_bajtow; // łączna liczba zajętych bajtów
    int magazyn_otwarty; // Flaga stanu magazynu (1=otwarty, 0=zamknięty)
} Magazyn;
```

Funkcja `inicjalizuj_magazyn()`:

- Zeruje wszystkie wskaźniki head/tail dla każdej kolejki
- Ustawia `count = 0` dla wszystkich kolejek
- Inicjalizuje `suma_bajtow = 0`
- Ustawia `magazyn_otwarty = 1` (domyślnie otwarty)
- Czyści bufor danych funkcją `memset()` dla bezpieczeństwa

3.2 Operacje na Kolejках FIFO

Procedura `wstaw_do_kolejki(Magazyn* mag, char typ)`:

Wstawia jeden składnik do odpowiedniej kolejki FIFO z zachowaniem kolejności:

1. **Walidacja:** Sprawdza czy kolejka nie jest pełna (`count < KOLEJKA_POJEMNOSC`)
2. **Kontrola limitu:** Weryfikuje czy nie przekroczono `MAGAZYN_POJEMNOSC` bajtów
3. **Zapis danych:** Wstawia bajt na pozycję `head` w tablicy cyklicznej
4. **Aktualizacja wskaźnika:** `head = (head + 1) % KOLEJKA_POJEMNOSC` (cykliczne przejście)
5. **Inkrementacja liczników:** Zwiększa `count` i `suma_bajtow`
6. **Zwrot wyniku:** Sukces (1) lub porażka (0)

Procedura `pobierz_z_kolejki(Magazyn* mag, char typ)`:

Pobiera najstarszy składnik z kolejki FIFO:

1. **Walidacja:** Sprawdza czy kolejka nie jest pusta (`count > 0`)
2. **Odczyt danych:** Pobiera bajt z pozycji `tail`
3. **Aktualizacja wskaźnika:** `tail = (tail + 1) % KOLEJKA_POJEMNOSC`
4. **Dekrementacja liczników:** Zmniejsza `count` i `suma_bajtow`

5. **Zwrot wartości:** Pobrany bajt reprezentujący składnik

3.3 Synchronizacja Semaforami

System wykorzystuje **11 semaforów** w hierarchicznej strukturze:

Semafor	Typ	Wartość początkowa	Rola
SEM_MUTEX	Binary	1	Mutex chroniący sekcje krytyczne
SEM_WOLNE	Counting	70	Licznik wolnych bajtów w magazynie
SEM_SKLAD_A/B/C/D	Counting	0	Dostępność składników (producer-consumer)
SEM_LIMIT_A/B/C/D	Counting	KOLEJKA_POJEMNOSC	Limit miejsc w kolejkach
SEM_LOG	Binary	1	Ochrona dostępu do pliku raportu

Funkcja `inicjalizuj_semafory(int sem_id)`:

- Tworzy zestaw 11 semaforów przez `semget()`
- Ustawia wartości początkowe przez `semctl()` z `SETVAL`
- Używa `union semun` dla przekazania wartości

3.4 Pamięć Dzielona (Shared Memory)

Procedura tworzenia i zarządzania:

```
int shm_id = shmget(SHM_KEY, sizeof(Magazyn), IPC_CREAT | 0600);
Magazyn* mag = shmat(shm_id, NULL, 0);
```

Cykl życia pamięci dzielonej:

1. **Utworzenie** (`utworz_pamiec_dzielona`): Dyrektor alokuje segment przez `shmget()`
2. **Dołączenie** (`polacz_z_pamiecia_dzielona`): Wszystkie procesy mapują przez `shmat()`
3. **Użycie**: Procesy czytają/piszą do współdzielonej struktury `Magazyn`
4. **Odłączenie** (`odlacz_pamiec_dzielona`): Każdy proces odpina przez `shmdt()`
5. **Usunięcie** (`usun_pamiec_dzielona`): Dyrektor usuwa przez `shmctl(IPC_RMID)`

Bezpieczeństwo dostępu:

- Każda operacja odczytu/zapisu chroniona przez `SEM_MUTEX`
- Sekcje krytyczne jak najkrótsze (minimalizacja blokowania)
- Brak aktywnego czekania (busy-waiting) - tylko semafony blokujące

3.5 Obsługa Sygnałów

SIGCHLD - Zapobieganie procesom zombie:

```
void handle_sigchld(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0); // Nieblokujące zbieranie
}
```

- Rejestracja: `sigaction()` z flagą `SA_RESTART`
- Handler wywoływany automatycznie przy zakończeniu potomka
- Pętla zbiera wszystkie zakończone procesy jednym wywołaniem
- Flaga `WNOHANG` zapobiega blokowaniu

SIGINT - (Ctrl+C):

```
void handle_sigint(int sig) {
    running = 0; // Ustawia flagę zakończenia
    // Propagacja SIGTERM do potomków
    for(int i=0; i<liczba_dostawcow; i++)
        kill(pids_dostawcy[i], SIGTERM);
    for(int i=0; i<liczba_pracownikow; i++)
        kill(pids_pracownicy[i], SIGTERM);
}
```

- Rejestracja: `sigaction()` **bez** `SA_RESTART` (przerywa `scanf`)

- Handler propaguje sygnał zakończenia do wszystkich potomków
- Dyrektor zapisuje stan magazynu przed zakończeniem
- Wszystkie procesy czyszczą zasoby IPC

SIGUSR1 i SIGUSR2 - Kontrola procesów:

- SIGUSR1 : Zatrzymanie pracowników (opcja 1 w menu)
- SIGUSR2 : Zatrzymanie dostawców (opcja 3 w menu)
- Procesy ustawiają `running = 0` i kończą główną pętlę
- Zakończenie procesów - odpinają pamięć przed zakończeniem

3.6 Atomowe Operacje Semaforowe

Funkcja `semop()` z wieloma operacjami:

Dostawca czeka atomowo na miejsce:

```
struct sembuf czekaj[2];
czekaj[0] = {SEM_WOLNE, -rozmiar, 0};    // Czekaj na bajty
czekaj[1] = {SEM_LIMIT_X, -1, 0};        // Czekaj na slot
semop(sem_id, czekaj, 2); // Atomowa operacja - wszystko albo nic
```

Pracownik pobiera atomowo komplet składników:

```
struct sembuf czekaj[3];
czekaj[0] = {SEM_SKLAD_A, -1, 0}; // A
czekaj[1] = {SEM_SKLAD_B, -1, 0}; // B
czekaj[2] = {SEM_SKLAD_C, -1, 0}; // C lub D
semop(sem_id, czekaj, 3); // Pobiera wszystko jednocześnie
```

Zalety atomowości:

- Zapobiega zakleszczeniom
- Gwarantuje kompletność operacji
- Eliminuje race conditions
- Upraszcza logikę synchronizacji

3.7 Logowanie z Ochroną przez Semafor

Procedura `wyslij_log(int sem_id, const char* tekst)`:

1. **Czekaj na dostęp:** `sem_wait(sem_id, SEM_LOG)`
2. **Sekcja krytyczna:**
 - Otwórz plik w trybie append: `fopen("raport.txt", "a")`
 - Pobierz timestamp: `time() + localtime()`
 - Formatuj wpis: `[YYYY-MM-DD HH:MM:SS] tekst\n`
 - Zapisz: `fprintf()`
 - Zamknij: `fclose()`
3. **Zwolnij dostęp:** `sem_signal(sem_id, SEM_LOG)`

Korzyści:

- Brak uszkodzenia logów (brak race condition)
- Chronologiczna kolejność wpisów
- Bezpieczny dostęp wieloprotocowy
- Timestampy dla debugowania

3.8 Bezpieczny Zapis i Odczyt Stanu Magazynu

Zapis stanu - `zapisz_stan_magazynu()`:

1. Otwiera plik binarny: `fopen(MAGAZYN_PLIK, "wb")`
2. Zapisuje całą strukturę: `fwrite(&magazyn, sizeof(Magazyn), 1, plik)`
3. Zamyka plik i zwraca status

Odczyt stanu - `odczytaj_stan_magazynu()`:

1. Sprawdza istnienie: `access(MAGAZYN_PLIK, F_OK)`
2. Otwiera plik: `fopen(MAGAZYN_PLIK, "rb")`
3. Wczytuje strukturę: `fread(&magazyn, sizeof(Magazyn), 1, plik)`
4. Zamyka plik

Aktualizacja semaforów - `zaktualizuj_semafony()`: Po wczytaniu stanu z pliku synchronizuje wartości semaforów:

- `SEM_WOLNE = MAGAZYN_POJEMNOSC - suma_bajtow`
- `SEM_SKLAD_X = count` dla każdej kolejki
- `SEM_LIMIT_X = KOLEJKA_POJEMNOSC - count`

3.9 Menu Interaktywne Dyrektora

5 opcji zarządzania:

1. **Polecenie_1:** Wysyła SIGUSR1 do pracowników (zatrzymanie fabryki)
2. **Polecenie_2:** Zmiana flagi magazyn_otwarty (zamknięcie/otwarcie magazynu)
3. **Polecenie_3:** Wysyła SIGUSR2 do dostawców (zatrzymanie dostaw)
4. **Polecenie_4:** Wysyła SIGTERM do wszystkich + zapis stanu + exit
5. **Polecenie_5:** Wyświetla stan magazynu (funkcja `wyswietl_stan_magazynu`)

Walidacja wejścia:

- Sprawdzanie flagi `running` przed i po `scanf()`
- Czyszczenie bufora wejścia przy błędzie
- Obsługa przerwania przez sygnał (SIGINT)

4. Pseudokody Kluczowych Algorytmów

4.1 Inicjalizacja Systemu (Dyrektor)

```
FUNKCJA main_dyrektor():
    // 1. Rejestracja handlerów sygnałów
    SIGACTION(SIGCHLD, handle_sigchld, SA_RESTART)
    SIGACTION(SIGINT, handle_sigint, BEZ_SA_RESTART)

    // 2. Inicjalizacja IPC
    shm_id = shmget(SHM_KEY, sizeof(Magazyn), IPC_CREAT | 0600)
    magazyn = shmat(shm_id, NULL, 0)

    // 3. Wczytanie lub inicjalizacja stanu
    JEŚLI istnieje_plik("magazyn_stan.dat"):
        odczytaj_stan_magazynu(magazyn, "magazyn_stan.dat")
        wczytano_stan = PRAWDA
    W_PRZECIWNYM_RAZIE:
        inicjalizuj_magazyn(magazyn)
        wczytano_stan = FAŁSZ
    KONIEC_JEŚLI

    // 4. Utworzenie semaforów
    sem_id = semget(SEM_KEY, 11, IPC_CREAT | 0600)
    inicjalizuj_semafony(sem_id)

    JEŚLI wczytano_stan:
        zaktualizuj_semafony(sem_id, magazyn)
    KONIEC_JEŚLI

    // 5. Uruchomienie dostawców
    DLA KAŻDEGO składnik W ['A', 'B', 'C', 'D']:
        pid = FORK()
        JEŚLI pid == 0: // Proces potomny
            EXECL("./bin/dostawca", składnik)
        W_PRZECIWNYM_RAZIE: // Proces rodzica
            pids_dostawcy[i] = pid
        KONIEC_JEŚLI
    KONIEC_DLA

    // 6. Uruchomienie pracowników
    DLA stanowisko OD 1 DO 2:
        pid = FORK()
        JEŚLI pid == 0: // Proces potomny
            EXECL("./bin/pracownik", stanowisko)
        W_PRZECIWNYM_RAZIE: // Proces rodzica
            pids_pracownicy[i] = pid
        KONIEC_JEŚLI
    KONIEC_DLA
```

```

// 7. Pętla menu
DOPÓKI running:
    wyświetl_menu()
    SCANF("%d", &opcja)

    JEŚLI !running:
        PRZERWIJ // Przerwano przez SIGINT
    KONIEC_JEŚLI

    SWITCH(opcja):
        CASE 1: stop_pracownikow()
        CASE 2: toggle_magazyn()
        CASE 3: stop_dostawcow()
        CASE 4: koniec_z_zapisem()
        CASE 5: wyświetl_stan()
    KONIEC_SWITCH
KONIEC_DOPÓKI

// 8. Czekanie na procesy potomne
DLA KAŻDEGO pid W pids_dostawcy:
    WAITPID(pid, NULL, 0)
KONIEC_DLA

DLA KAŻDEGO pid W pids_pracownicy:
    WAITPID(pid, NULL, 0)
KONIEC_DLA

// 9. Sprzątanie zasobów
shmdt(magazyn)
shmctl(shm_id, IPC_RMID, NULL)
semctl(sem_id, 0, IPC_RMID)

ZWRÓĆ 0
KONIEC_FUNKCJI

```

4.2 Dostawca - Dostarczanie Składnika

```

FUNKCJA main_dostawca(typ_skladnika):
    // 1. Rejestracja sygnałów
    SIGNAL(SIGUSR2, handle_signal) // Stop od dyrektora
    SIGNAL(SIGTERM, handle_signal) // Natychmiastowe zakończenie
    SIGNAL(SIGINT, handle_signal) // Ctrl+C

    // 2. Połączenie z IPC
    shm_id = shmget(SHM_KEY, 0, 0)
    magazyn = shmat(shm_id, NULL, 0)
    sem_id = semget(SEM_KEY, 0, 0)

    // 3. Określenie parametrów składnika
    rozmiar = ROZMIAR_SKLADNIKA[typ_skladnika]
    sem_skladnik = SEM_SKLAD[typ_skladnika]
    sem_limit = SEM_LIMIT[typ_skladnika]

    // 4. Główna pętla dostawcy
    DOPÓKI running:
        // Sprawdzenie czy magazyn otwarty
        JEŚLI !magazyn.magazyn_otwarty:
            LOG("Magazyn zamknięty - czekam...")
            SLEEP(2)
            KONTYNUUJ
        KONIEC_JEŚLI

        // Przygotowanie atomowej operacji semop
        operacje[0] = {SEM_WOLNE, -rozmiar, 0} // Czekaj na bajty
        operacje[1] = {sem_limit - 1, 0} // Czekaj na slot

```

```

operacje[1] = {sem_limit, -1, 0} // Czekaj na slot

// Sprawdzenie dostępności miejsca (opcjonalne logowanie)
wolne_bajty = SEMCTL(sem_id, SEM_WOLNE, GETVAL)
wolne_sloty = SEMCTL(sem_id, sem_limit, GETVAL)

JEŚLI wolne_bajty < rozmiar LUB wolne_sloty < 1:
    LOG("Brak miejsca w magazynie. Czekam...")
KONIEC_JEŚLI

// Atomowe czekanie na miejsce (blokujące)
wynik = SEMOP(sem_id, operacje, 2)
JEŚLI wynik == -1 I !running:
    PRZERWIJ // Przerwano sygnałem
KONIEC_JEŚLI

// ===== SEKCJA KRYTYCZNA START =====
SEM_WAIT(sem_id, SEM_Mutex)

// Sprawdzenie flagi running (może zmienić się podczas czekania)
JEŚLI !running:
    SEM_SIGNAL(sem_id, SEM_Mutex)
    PRZERWIJ
KONIEC_JEŚLI

// Wstawienie składnika do kolejki
sukces = wstaw_do_kolejki(magazyn, typ_skladnika)

JEŚLI sukces:
    LOG("Dostarczono 1 x " + typ_skladnika +
        " | Magazyn: " + magazyn.suma_bajtow + "/" + MAX)
KONIEC_JEŚLI

SEM_SIGNAL(sem_id, SEM_Mutex)
// ===== SEKCJA KRYTYCZNA KONIEC =====

// Sygnalizacja dostępności składnika (V operacja)
operacja_signal = {sem_skladnik, +1, 0}
SEMOP(sem_id, operacja_signal, 1)

// Losowy czas między dostawami
SLEEP(LOSOWA(1, 3))
KONIEC_DOPÓKI

// 5. Sprzątanie
LOG("Koniec pracy")
shmdt(magazyn)
ZWRÓĆ 0
KONIEC_FUNKCJI

```

4.3 Pracownik - Produkcja Czekolady

```

FUNKCJA main_pracownik(stanowisko):
    // 1. Rejestracja sygnałów
    SIGNAL(SIGUSR1, handle_signal) // Stop od dyrektora
    SIGNAL(SIGTERM, handle_signal)
    SIGNAL(SIGINT, handle_signal)

    // 2. Połączenie z IPC
    shm_id = shmget(SHM_KEY, 0, 0)
    magazyn = shmat(shm_id, NULL, 0)
    sem_id = semget(SEM_KEY, 0, 0)

    // 3. Określenie typu czekolady
    JEŚLI stanowisko == 1:

```

```

    typ = "TYP_1 (A+B+C)"
    skladniki = ['A', 'B', 'C']
    sem_trzeci = SEM_SKLAD_C
    sem_limit_trzeci = SEM_LIMIT_C
    rozmiar_trzeci = ROZMIAR_C
W_PRZECIWNYM_RAZIE:
    typ = "TYP_2 (A+B+D)"
    skladniki = ['A', 'B', 'D']
    sem_trzeci = SEM_SKLAD_D
    sem_limit_trzeci = SEM_LIMIT_D
    rozmiar_trzeci = ROZMIAR_D
KONIEC_JESLI

wyprodukowano = 0

// 4. Przygotowanie atomowej operacji pobierania składników
czekaj[0] = {SEM_SKLAD_A, -1, 0}
czekaj[1] = {SEM_SKLAD_B, -1, 0}
czekaj[2] = {sem_trzeci, -1, 0}

// 5. Główna pętla producenta czekolady
DOPÓKI running:
    // Sprawdzenie czy magazyn otwarty
    JEŚLI !magazyn.magazyn_otwarty:
        LOG("Magazyn zamknięty - czekam...")
        SLEEP(2)
        KONTYNUUJ
    KONIEC_JESLI

    LOG("Czekam na komplet składników...")

    // Atomowe czekanie na wszystkie 3 składniki (P operacja)
    wynik = SEMOP(sem_id, czekaj, 3)
    JEŚLI wynik == -1 I !running:
        PRZERWIJ
    KONIEC_JESLI

    // ===== SEKCJA KRYTYCZNA START =====
    SEM_WAIT(sem_id, SEM_Mutex)

    JEŚLI !running:
        SEM_SIGNAL(sem_id, SEM_Mutex)
        PRZERWIJ
    KONIEC_JESLI

    // Pobranie fizycznych składników z kolejek
    a = pobierz_z_kolejki(magazyn, 'A')
    b = pobierz_z_kolejki(magazyn, 'B')

    JEŚLI stanowisko == 1:
        c_d = pobierz_z_kolejki(magazyn, 'C')
        zwolnione_bajty = ROZMIAR_A + ROZMIAR_B + ROZMIAR_C
    W_PRZECIWNYM_RAZIE:
        c_d = pobierz_z_kolejki(magazyn, 'D')
        zwolnione_bajty = ROZMIAR_A + ROZMIAR_B + ROZMIAR_D
    KONIEC_JESLI

    LOG("Pobrano: " + a + ", " + b + ", " + c_d +
        " | Magazyn: " + magazyn.suma_bajtow + "/" + MAX)

    SEM_SIGNAL(sem_id, SEM_Mutex)
    // ===== SEKCJA KRYTYCZNA KONIEC =====

    // Atomowe zwolnienie miejsca w magazynie (V operacje)
    zwolnij[0] = {SEM_WOLNE, +zwolnione_bajty, 0}

```



```

    zwolnij[1] = {SEM_LIMIT_A, +1, 0}
    zwolnij[2] = {SEM_LIMIT_B, +1, 0}
    zwolnij[3] = {sem_limit_trzeci, +1, 0}

    SEMOP(sem_id, zwolnij, 4)

    // Produkcja czekolady (symulacja pracy)
    SLEEP(LOSOWA(1, 5))

    wyprodukowano++
    LOG("*** WYPRODUKOWANO CZEKOLADĘ - " + typ + " #" + wyprodukowano + " ***")
KONIEC_DOPÓKI

// 6. Raport końcowy
LOG("Koniec. Wyprodukowano: " + wyprodukowano + " czekolady " + typ)
shmdt(magazyn)
ZWRÓĆ 0
KONIEC_FUNKCJI

```

4.4 Ring Buffer FIFO - Operacje Cykliczne

```

FUNKCJA wstaw_do_kolejki(magazyn, typ):
    // 1. Pobranie wskaźnika do właściwej kolejki
    SWITCH(typ):
        CASE 'A': kolejka = &magazyn.kolejka_A; rozmiar = ROZMIAR_A
        CASE 'B': kolejka = &magazyn.kolejka_B; rozmiar = ROZMIAR_B
        CASE 'C': kolejka = &magazyn.kolejka_C; rozmiar = ROZMIAR_C
        CASE 'D': kolejka = &magazyn.kolejka_D; rozmiar = ROZMIAR_D
        DEFAULT: ZWRÓĆ FAŁSZ
    KONIEC_SWITCH

    // 2. Walidacja limitów
    JEŚLI kolejka.count >= KOLEJKA_POJEMNOSC:
        ZWRÓĆ FAŁSZ // Kolejka pełna
    KONIEC_JEŚLI

    JEŚLI magazyn.suma_bajtow + rozmiar > MAGAZYN_POJEMNOSC:
        ZWRÓĆ FAŁSZ // Przekroczono limit bajtów
    KONIEC_JEŚLI

    // 3. Wstawienie danych na pozycję head
    kolejka.dane[kolejka.head] = typ

    // 4. Przesunięcie wskaźnika head (cyklicznie)
    kolejka.head = (kolejka.head + 1) % KOLEJKA_POJEMNOSC

    // 5. Aktualizacja liczników
    kolejka.count++
    magazyn.suma_bajtow += rozmiar

    ZWRÓĆ PRAWDA
KONIEC_FUNKCJI

FUNKCJA pobierz_z_kolejki(magazyn, typ):
    // 1. Pobranie wskaźnika do właściwej kolejki
    kolejka = pobierz_kolejke_dla_typu(magazyn, typ)
    JEŚLI kolejka == NULL:
        ZWRÓĆ 0
    KONIEC_JEŚLI

    // 2. Sprawdzenie czy kolejka nie jest pusta
    JEŚLI kolejka.count <= 0:
        ZWRÓĆ 0 // Kolejka pusta
    KONIEC_JEŚLI

```

```

    rozmiar = rozmiar_skladnika(typ)

    // 3. Pobranie danych z pozycji tail
    bajt = kolejka.dane[kolejka.tail]

    // 4. Przesunięcie wskaźnika tail (cyklicznie)
    kolejka.tail = (kolejka.tail + 1) % KOLEJKA_POJEMNOSC

    // 5. Aktualizacja liczników
    kolejka.count--
    magazyn.suma_bajtow -= rozmiar

    ZWRÓĆ bajt
KONIEC_FUNKCJI

FUNKCJA inicjalizuj_kolejke(kolejka):
    kolejka.head = 0
    kolejka.tail = 0
    kolejka.count = 0

    // Wyzerowanie bufora danych
    DLA i OD 0 DO KOLEJKA_POJEMNOSC-1:
        kolejka.dane[i] = 0
    KONIEC_DLA
KONIEC_FUNKCJI

```

4.5 Zapis i Odczyt stanu magazynu do pliku

```

FUNKCJA zapisz_stan_magazynu(magazyn, nazwa_pliku):
    // 1. Otwarcie pliku w trybie binarnym
    plik = FOPEN(nazwa_pliku, "wb")
    JEŚLI plik == NULL:
        LOG("Błąd: Nie można otworzyć pliku do zapisu")
        ZWRÓĆ -1
    KONIEC_JEŚLI

    // 2. Zapis całej struktury magazynu (binarnie)
    zapisane = FWRITE(&magazyn, sizeof(Magazyn), 1, plik)

    // 3. Zamknięcie pliku
    FCLOSE(plik)

    // 4. Weryfikacja zapisu
    JEŚLI zapisane != 1:
        LOG("Błąd: Nie udało się zapisać danych")
        ZWRÓĆ -1
    KONIEC_JEŚLI

    LOG("Stan magazynu zapisany do: " + nazwa_pliku)
    ZWRÓĆ 0
KONIEC_FUNKCJI

FUNKCJA odczytaj_stan_magazynu(magazyn, nazwa_pliku):
    // 1. Sprawdzenie istnienia pliku
    JEŚLI !ACCESS(nazwa_pliku, F_OK):
        LOG("Plik nie istnieje: " + nazwa_pliku)
        ZWRÓĆ -1
    KONIEC_JEŚLI

    // 2. Otwarcie pliku w trybie binarnym
    plik = FOPEN(nazwa_pliku, "rb")
    JEŚLI plik == NULL:
        LOG("Błąd: Nie można otworzyć pliku do odczytu")
        ZWRÓĆ -1

```

```

KONIEC_JEŚLI

// 3. Odczyt całej struktury magazynu
wczytane = FREAD(&magazyn, sizeof(Magazyn), 1, plik)

// 4. Zamknięcie pliku
FCLOSE(plik)

// 5. Weryfikacja odczytu
JEŚLI wczytane != 1:
    LOG("Błąd: Nie udało się wczytać danych")
    ZWRÓĆ -1
KONIEC_JEŚLI

LOG("Stan magazynu wczytany z: " + nazwa_pliku)
ZWRÓĆ 0
KONIEC_FUNKCJI

FUNKCJA zaktualizuj_semafony(sem_id, magazyn):
    // Po wczytaniu stanu synchronizuj wartości semaforów

    // 1. Zaktualizuj SEM_WOLNE (wolne bajty)
    wolne_bajty = MAGAZYN_POJEMNOSC - magazyn.suma_bajtow
    SEMCTL(sem_id, SEM_WOLNE, SETVAL, wolne_bajty)

    // 2. Zaktualizuj dostępność składników (SEM_SKLAD_X)
    SEMCTL(sem_id, SEM_SKLAD_A, SETVAL, magazyn.kolejka_A.count)
    SEMCTL(sem_id, SEM_SKLAD_B, SETVAL, magazyn.kolejka_B.count)
    SEMCTL(sem_id, SEM_SKLAD_C, SETVAL, magazyn.kolejka_C.count)
    SEMCTL(sem_id, SEM_SKLAD_D, SETVAL, magazyn.kolejka_D.count)

    // 3. Zaktualizuj limity slotów (SEM_LIMIT_X)
    SEMCTL(sem_id, SEM_LIMIT_A, SETVAL, KOLEJKA_POJEMNOSC - magazyn.kolejka_A.count)
    SEMCTL(sem_id, SEM_LIMIT_B, SETVAL, KOLEJKA_POJEMNOSC - magazyn.kolejka_B.count)
    SEMCTL(sem_id, SEM_LIMIT_C, SETVAL, KOLEJKA_POJEMNOSC - magazyn.kolejka_C.count)
    SEMCTL(sem_id, SEM_LIMIT_D, SETVAL, KOLEJKA_POJEMNOSC - magazyn.kolejka_D.count)

    LOG("Semafony zaktualizowane zgodnie ze stanem magazynu")
KONIEC_FUNKCJI

```

4.6 Logowanie z Ochroną przez Semafor

```

FUNKCJA wyslij_log(sem_id, tekst):
    // 1. Czekanie na dostęp do pliku logu
    JEŚLI sem_id >= 0:
        SEM_WAIT(sem_id, SEM_LOG)
    KONIEC_JEŚLI

    // ===== SEKCJA KRYTYCZNA START =====

    // 2. Otwarcie pliku w trybie append
    plik = FOPEN("raport.txt", "a")
    JEŚLI plik == NULL:
        JEŚLI sem_id >= 0:
            SEM_SIGNAL(sem_id, SEM_LOG)
        KONIEC_JEŚLI
        ZWRÓĆ
    KONIEC_JEŚLI

    // 3. Pobranie aktualnego czasu
    czas = TIME(NULL)
    czas_lokalny = LOCALTIME(&czas)

    // 4. Formatowanie timestampu
    STRFTIME(timestamp, 64, "[%Y-%m-%d %H:%M:%S]", czas_lokalny)

    // 5. Zapis wpisu do pliku
    FPRINTF(plik, "%s %s\n", timestamp, tekst)

    // 6. Zamknięcie pliku
    FCLOSE(plik)

    // 7. Wyświetlenie również na stdout
    PRINTF("%s\n", tekst)

    // ===== SEKCJA KRYTYCZNA KONIEC =====

    // 8. Zwolnienie dostępu do pliku
    JEŚLI sem_id >= 0:
        SEM_SIGNAL(sem_id, SEM_LOG)
    KONIEC_JEŚLI
KONIEC_FUNKCJI

```

5. Problemy Napotkane w Trakcie Realizacji

5.1 Problem z Procesami Zombie

Problem: Po zakończeniu procesów potomnych pozostawały procesy zombie.

Rozwiązanie: Implementacja handlera `SIGCHLD` z pętlą `waitpid(-1, NULL, WNOHANG)` i flagą `SA_RESTART` w `sigaction`.

5.2 Przerwanie `scanf()` przez Ctrl+C

Problem: Handler `SIGINT` z flagą `SA_RESTART` nie przerywał `scanf()`.

Rozwiązanie: Użycie `sigaction` bez flagi `SA_RESTART` dla `SIGINT`, aby przerwać blokujące wywołania systemowe.

5.3 Race Condition w Dostępie do Pliku

Problem: Wielokrotny dostęp do `raport.txt` powodował uszkodzenie logów.

Rozwiązanie: Dedykowany semafor `SEM_LOG` chroniący operacje I/O na pliku.

5.4 Zakleszczenie Pracowników

Problem: Pracownicy czekali na składniki, które nie mogły być dostarczone.

Rozwiązanie: Atomowe operacje `semop()` z 3 semaforami jednocześnie dla kompletu składników.

6. Dodane Elementy

6.1 Dynamiczne Obliczanie Pojemności Kolejek

Zamiast sztywnej wartości, pojemność jest obliczana na podstawie pojemności magazynu oraz rozmiaru składników:

```
#define SUMA_ROZMIAROW (ROZMIAR_A + ROZMIAR_B + ROZMIAR_C + ROZMIAR_D)
#define KOLEJKA_POJEMNOSC ((MAGAZYN_POJEMNOSC + SUMA_ROZMIAROW - 1) / SUMA_ROZMIAROW)
```

6.2 Kolorowe Logi w Terminalu

Użycie sekwencji ANSI dla lepszej czytelności:

```
#define KOLOR_RESET      "\033[0m"
#define KOLOR_CZERWONY  "\033[31m"
#define KOLOR_ZIELONY   "\033[32m"
#define KOLOR_ZOLTY     "\033[33m"
#define KOLOR_NIEBIESKI "\033[34m"
#define KOLOR_MAGENTA   "\033[35m"
#define KOLOR_CYAN      "\033[36m"
#define KOLOR_BIALY     "\033[37m"
#define KOLOR_BOLD      "\033[1m"
```

6.3 Automatyczny zapis stanu magazynu przy CTRL+C

Automatyczny zapis stanu magazynu dodatkowo przy Ctrl+C :

- Przy przerwaniu przez Ctrl+C
- Przy uruchomieniu odczytuje poprzedni stan

6.4 Ochrona przed Zamknięciem Magazynu

Opcja 2 w menu pozwala tymczasowo zamknąć/otworzyć magazyn bez kończenia procesów - dostawcy i pracownicy czekają w pętli.

7. Testy

7.1 TEST 1: Weryfikacja Sprzątania Zasobów IPC

Cel: Potwierdzenie, że po zakończeniu programu (Opcja 4) w systemie nie pozostają wiszące semaforey ani segmenty pamięci.

Scenariusz:

1. Uruchom Dyrektora.
2. W **Terminalu B** sprawdź zasoby: `ipcs`. Powinieneś widzieć 1 segment SHM i 1 tablicę SEM.
3. W **Terminalu A** wybierz **Opcję 4** (Koniec + zapis).
4. Ponownie wpisz `ipcs` w **Terminalu B**.

Oczekiwany rezultat:

- Lista `ipcs` po zakończeniu programu jest czysta.

Weryfikacja testu:

Praca magazynu przed wybraniem opcji "4"

```
[23:24:36] [PRACOWNIK-2] Czekam na komplet skladnikow...
[23:24:36] [PRACOWNIK-2] Pobrano: A, B, D | Magazyn zajety: 8/70 |
[23:24:36] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 10/70 |
[23:24:36] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 13/70 |
```

Podgląd w terminalu B na procesy `-ipcs`

```

----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages

----- Shared Memory Segments -----
key          shmid      owner      perms      bytes       nattch     status
0x00001234  0                kamil      600        104         7

----- Semaphore Arrays -----
key          semid      owner      perms      nsems
0x00005678  0                kamil      600        11

```

Wybranie opcji "4"

```

4
[23:24:36] >> [DYREKTOR] Koniec symulacji. Zatrzymuje wszystkich...
[23:24:36] [DOSTAWCA-A] Koniec pracy
[23:24:36] [DOSTAWCA-B] Koniec pracy
[23:24:36] [DOSTAWCA-C] Koniec pracy
[23:24:36] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #2 ***
[23:24:36] [DOSTAWCA-D] Koniec pracy
[PLIK] Zapisano stan. Magazyn zajety: 13/70 (A:0 B:0 C:2 D:3)
[23:24:36] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #2 ***
[23:24:36]
[SHM] Odłączono od pamięci dzielonej
[DYREKTOR] Przerwano program - zapisuje stan magazynu...
[SHM] Odłączono od pamięci dzielonej
[23:24:36] [PRACOWNIK-2] Koniec. Wyprodukowano: 2 czekolady TYP_2 (A+B+D)
[PLIK] Zapisano stan. Magazyn zajety: 13/70 (A:0 B:0 C:2 D:3)
[23:24:36] [DYREKTOR] Czekam na zakończenie procesów potomnych...
[SHM] Odłączono od pamięci dzielonej
[SHM] Odłączono od pamięci dzielonej
[23:24:36] [PRACOWNIK-1] Koniec. Wyprodukowano: 2 czekolady TYP_1 (A+B+C)
[SHM] Odłączono od pamięci dzielonej
[SHM] Odłączono od pamięci dzielonej
[23:24:36] [DYREKTOR] Wszystkie procesy potomne zakończone

```

Ponowny podgląd w terminalu B na procesy `-ipcs` po zakończeniu programu

```

----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages

----- Shared Memory Segments -----
key          shmid      owner      perms      bytes       nattch     status

----- Semaphore Arrays -----
key          semid      owner      perms      nsems

```

Rezultat testu jest zgodny z oczekiwaniami:

- Lista `ipcs` po zakończeniu programu jest czysta. Test zaliczony.

7.2 TEST 2: Obsługa sygnału CTRL+C

Cel: Sprawdzenie poprawności obsługi sygnału `SIGINT` (Ctrl+C), sprawdzenie czy zasoby `ipcs` zostały wyczyszczone, zapisu stanu magazynu do pliku oraz jego poprawnego odtworzenia po restarcie.

Scenariusz:

1. W **Terminalu A** uruchom: `./bin/dyrektor`.
2. Poczekaj, aż magazyn zapełni się w ok. 30-50%.
3. Wciśnij Ctrl+C w **Terminalu A**.
4. W **Terminalu B** sprawdź czy zasoby `ipcs` zostały wyczyszczone
5. Ponownie uruchom `./bin/dyrektor` w **Terminalu A** i zobacz czy poprawnie wczytano stan magazynu.

Oczekiwany rezultat:

- Program kończy się komunikatem o zapisie stanu (np. `[DYREKTOR] Przerwano program - zapisuje stan magazynu...`).
- Lista `ipcs` po zakończeniu programu jest czysta.

- Po ponownym uruchomieniu w logach pojawia się: [DYREKTOR] Stan magazynu odtworzony z pliku! .
- Liczba surowców i zajętość magazynu jest identyczna jak w momencie zamknięcia.

Weryfikacja testu:

Praca magazynu przed CTRL+C:

```
23:41:40] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 32/70 |
[23:41:40] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 34/70 |
[23:41:40] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #6 ***
[23:41:40] [PRACOWNIK-1] Czekam na komplet składników...
[23:41:40] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 30/70 |
[23:41:41] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #5 ***
[23:41:41] [PRACOWNIK-2] Czekam na komplet składników...
[23:41:41] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 31/70 |
```

Sygnal CTRL+C oraz stan magazynu po sygnale:

```
[23:41:42] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 35/70 |
[23:41:42] [PRACOWNIK-2] Pobrano: A, B, D | Magazyn zajety: 30/70 |
^C[23:41:43] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #6 ***
[23:41:43] [DOSTAWCA-A] Koniec pracy

[DYREKTOR] Otrzymano Ctrl+C - kończę pracę...
[23:41:43] [DOSTAWCA-C] Koniec pracy
[23:41:43]
[DYREKTOR] Przerwano program - zapisuje stan magazynu...
[23:41:43] [DOSTAWCA-D] Koniec pracy
[23:41:43] [DOSTAWCA-B] Koniec pracy
[23:41:43] [PRACOWNIK-2] Koniec. Wyprodukowano: 6 czekolady TYP_2 (A+B+D)
...
[23:41:43] +-----+
[23:41:43] |                STAN MAGAZYNU                |
[23:41:43] +-----+
[23:41:43] | Składnik A:   2 szt. ( 2 bajtów)                |
[23:41:43] | Składnik B:   0 szt. ( 0 bajtów)                |
[23:41:43] | Składnik C:   5 szt. ( 10 bajtów)               |
[23:41:43] | Składnik D:   6 szt. ( 18 bajtów)               |
[23:41:43] +-----+
[23:41:43] | Zajete:   30 / 70 bajtów ( 42%)                 |
[23:41:43] | Wolne:    40 bajtów                             |
[23:41:43] +-----+
```

Podgląd w terminalu B na procesy -ipcs :

```
----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages

----- Shared Memory Segments -----
key          shmid      owner      perms      bytes       nattch     status

----- Semaphore Arrays -----
key          semid      owner      perms      nsems
```

Ponowne uruchomienie programu:

```
23:45:57] [DYREKTOR] Inicjalizacja magazynu...
[23:45:57] [DYREKTOR] Znaleziono zapisany stan magazynu
[PLIK] Odczytano i zweryfikowano stan. Zajete: 30/70
[23:45:57] [DYREKTOR] Stan magazynu odtworzony z pliku!
[SEM] Utworzono zestaw semaforow (ID: 4, liczba: 11)
[23:45:57]
[23:45:57] [DYREKTOR] Poczatkowy stan magazynu:
[23:45:57]
[23:45:57] +-----+
[23:45:57] |                STAN MAGAZYNU                |
[23:45:57] +-----+
[23:45:57] | Skladnik A:   2 szt. (  2 bajtow)                |
[23:45:57] | Skladnik B:   0 szt. (  0 bajtow)                |
[23:45:57] | Skladnik C:   5 szt. ( 10 bajtow)                |
[23:45:57] | Skladnik D:   6 szt. ( 18 bajtow)                |
[23:45:57] +-----+
[23:45:57] | Zajete:    30 /   70 bajtow ( 42%)                |
[23:45:57] | Wolne:     40 bajtow                             |
[23:45:57] +-----+
[23:45:57]
```

7.3 TEST 3: Obsługa polecenia "1" (Stop Pracowników) i Weryfikacja Zombie

Cel: Sprawdzenie, czy Pracownicy kończą pracę na żądanie, czy Dostawcy dopełniają magazyn i czy nie powstają procesy zombie.

Scenariusz:

1. Uruchom program
2. Wybierz z menu **Opcję 1** (Fabryka kończy pracę).
3. Obserwuj logi – Pracownicy powinni przestać pobierać towary.
4. Poczekaj, aż Dostawcy zapełnią magazyn do 100% (lub limitów).
5. Wciśnij CTRL+Z i sprawdź ps czy są procesy pracownik

Oczekiwany rezultat:

- Magazyn osiąga pełną pojemność.
- Dostawcy przechodzą w stan oczekiwania ("Brak miejsca").
- Komenda ps **nie zwraca** żadnych procesów pracownik (brak procesów zombie).

Weryfikacja testu:


```
[23:49:58] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 8/70 |
[23:49:58] [PRACOWNIK-2] Pobrano: A, B, D | Magazyn zajety: 3/70 |
[23:49:59] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 6/70 |
[23:49:59] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #1 ***
[23:49:59] [PRACOWNIK-1] Czekam na komplet skladnikow...
1
[23:49:59] >> [DYREKTOR] Wysylam SIGUSR1 do Pracownikow...

Wybierz opcje: [23:49:59] [PRACOWNIK-1] Koniec. Wyprodukowano: 1 czekolady TYP_1 (A+B+C)
[23:49:59] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #1 ***
[SHM] Odloczono od pamieci dzielonej
[23:49:59] [PRACOWNIK-2] Koniec. Wyprodukowano: 1 czekolady TYP_2 (A+B+D)
[SHM] Odloczono od pamieci dzielonej
...
[23:50:13] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 59/70 |
[23:50:13] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 61/70 |
[23:50:13] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 64/70 |
[23:50:14] [DOSTAWCA-D] Brak miejsca w magazynie. Czekam...
[23:50:15] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 65/70 |
[23:50:16] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 66/70 |
[23:50:16] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 67/70 |
[23:50:16] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 69/70 |
[23:50:17] [DOSTAWCA-B] Brak miejsca w magazynie. Czekam...
[23:50:18] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 70/70 |
[23:50:18] [DOSTAWCA-C] Brak miejsca w magazynie. Czekam...
[23:50:21] [DOSTAWCA-A] Brak miejsca w magazynie. Czekam...
^Z
[1]+  Stopped                  ./bin/dyrektor
...
kamil      13184  0.0  0.0   2700  1664 pts/4    T   23:49   0:00 ./bin/dyrektor
kamil      13185  0.0  0.0   2692  1536 pts/4    T   23:49   0:00 dostawca A
kamil      13186  0.0  0.0   2692  1536 pts/4    T   23:49   0:00 dostawca B
kamil      13187  0.0  0.0   2692  1536 pts/4    T   23:49   0:00 dostawca C
kamil      13188  0.0  0.0   2692  1536 pts/4    T   23:49   0:00 dostawca D
kamil      14331  0.0  0.0   8284  4096 pts/4    R+  23:53   0:00 ps ux
```

Rezultat testu jest zgodny z oczekiwaniami:

- Pracownicy odbierają sygnał od dyrektora i kończą pracę. Dostawcy dopełniają magazyn do 100% pojemności oraz brak procesów pracowników po wpisaniu komendy `ps ux`. Test zaliczony.

7.4 TEST 4: Obsługa Sygnału 3 (Stop Dostawców) i Opróżnianie Magazynu

Cel: Sprawdzenie, czy Dostawcy kończą pracę na żądanie, a Pracownicy zużywają pozostałe zasoby do zera.

Scenariusz:

1. Uruchom program
2. Wybierz z menu **Opcję 3** (Dostawcy przerywają pracę).
3. Obserwuj logi – Dostawcy przestają dostarczać towar.
4. Pracownicy kontynuują pracę, zużywając zapasy.
5. Wciśnij CTRL+Z i sprawdź ps czy są procesy dostawca

Oczekiwany rezultat:

- Stan magazynu spada do 0 bajtów (lub blisko 0, jeśli brakuje kompletu do ostatniej produkcji).
- Pracownicy wyświetlają logi: Czekam na komplet skladnikow... .
- Komenda `ps` **nie zwraca** żadnych procesów dostawca (brak procesów zombie).

Weryfikacja testu:

Wysłanie polecenia "3" przez dyrektora:

```
[00:05:25] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 60/70 |
[00:05:25] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 61/70 |
[00:05:26] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #8 ***
[00:05:26] [PRACOWNIK-1] Czekam na komplet skladnikow...
[00:05:26] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 57/70 |
[00:05:26] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 59/70 |
[00:05:26] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 60/70 |
[00:05:27] [DOSTAWCA-D] Brak miejsca w magazynie. Czekam...
[00:05:27] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 61/70 |
[00:05:28] [DOSTAWCA-C] Brak miejsca w magazynie. Czekam...
[00:05:28] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 62/70 |
3
[00:05:29] >> [DYREKTOR] Wysylam SIGUSR2 do Dostawcow...
```

```
[DOSTAWCA-A] Koniec pracy
[00:05:29] [DOSTAWCA-C] Koniec pracy
[SHM] Odloczono od pamieci dzielonej
[00:05:29] [DOSTAWCA-B] Koniec pracy
[SHM] Odloczono od pamieci dzielonej
[00:05:29] [DOSTAWCA-D] Koniec pracy
[SHM] Odloczono od pamieci dzielonej
[SHM] Odloczono od pamieci dzielonej
```

Pracownicy produkują czekoladę dopóki mają odpowiednią ilość składników w magazynie

```
[00:05:36] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 53/70 |
[00:05:42] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #10 ***
[00:05:42] [PRACOWNIK-1] Czekam na komplet skladnikow...
[00:05:42] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 49/70 |
[00:05:42] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #12 ***
[00:05:42] [PRACOWNIK-2] Czekam na komplet skladnikow...
[00:05:42] [PRACOWNIK-2] Pobrano: A, B, D | Magazyn zajety: 44/70 |
[00:05:44] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #11 ***
[00:05:44] [PRACOWNIK-1] Czekam na komplet skladnikow...
[00:05:44] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 40/70 |
[00:05:47] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #12 ***
[00:05:47] [PRACOWNIK-1] Czekam na komplet skladnikow...
[00:05:52] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #13 ***
[00:05:52] [PRACOWNIK-2] Czekam na komplet skladnikow...
5
[00:07:39]
[00:07:39] +-----+
[00:07:39] |                STAN MAGAZYNU                |
[00:07:39] +-----+
[00:07:39] | Skladnik A:   2 szt. (  2 bajtow)                |
[00:07:39] | Skladnik B:   0 szt. (  0 bajtow)                |
[00:07:39] | Skladnik C:   7 szt. ( 14 bajtow)                |
[00:07:39] | Skladnik D:   8 szt. ( 24 bajtow)                |
[00:07:39] +-----+
[00:07:39] | Zajete:   40 /   70 bajtow ( 57%)                |
[00:07:39] | Wolne:    30 bajtow                             |
[00:07:39] +-----+
```

Sprawdzenie czy są procesy dostawców po wysłaniu polecenia "3":

kamil	17512	0.0	0.0	2700	1664	pts/4	T	00:04	0:00	./bin/dyrektor
kamil	17517	0.0	0.0	2692	1536	pts/4	T	00:04	0:00	pracownik 1
kamil	17518	0.0	0.0	2692	1408	pts/4	T	00:04	0:00	pracownik 2
kamil	18860	0.0	0.0	8284	4096	pts/4	R+	00:08	0:00	ps ux

Rezultat testu jest zgodny z oczekiwaniami:

- Pracownicy odbierają sygnał od dyrektora i kończą pracę. Pracownicy produkują czekoladę dopóki mają wystarczającą ilość składników w magazynie oraz brak procesów dostawców po wpisaniu komendy `ps ux`. Test zaliczony.

7.5 TEST 5: Zamknięcie i otwarcie magazynu

Cel: Weryfikacja, czy zamknięcie magazynu przez Dyrektora wstrzymuje operacje obu grup procesów bez ich zabijania.

Scenariusz:

1. Uruchom system w normalnym trybie.
2. Wybierz **Opcję 2** (Zamknij Magazyn).
3. Odczekaj 10 sekund.
4. Wybierz ponownie **Opcję 2** (Otwórz Magazyn).

Oczekiwany rezultat:

- Po zamknięciu, procesy logują: Magazyn zamknięty - czekam...
- Stan magazynu (liczba bajtów) nie zmienia się w trakcie blokady.
- Po ponownym otwarciu procesy natychmiast wracają do logowania produkcji i dostaw.

Weryfikacja testu

Uruchomienie programu i wysłanie przez dyrektora polecenia "2" - zamknięcie magazynu

```
00:18:34] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 48/70 |
[00:18:34] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 50/70 |
[00:18:35] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #10 ***
[00:18:35] [PRACOWNIK-1] Czekam na komplet składników...
[00:18:35] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 46/70 |
[00:18:35] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 47/70 |
[00:18:35] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #10 ***
[00:18:35] [PRACOWNIK-2] Czekam na komplet składników...
[00:18:35] [PRACOWNIK-2] Pobrano: A, B, D | Magazyn zajety: 42/70 |
2
[00:18:36]
[00:18:36] +-----+
[00:18:36] |                STAN MAGAZYNU                |
[00:18:36] +-----+
[00:18:36] | Składnik A:   0 szt. (  0 bajtów)              |
[00:18:36] | Składnik B:   0 szt. (  0 bajtów)              |
[00:18:36] | Składnik C:   9 szt. ( 18 bajtów)              |
[00:18:36] | Składnik D:   8 szt. ( 24 bajtów)              |
[00:18:36] +-----+
[00:18:36] | Zajete:    42 /   70 bajtów ( 60%)              |
[00:18:36] | Wolne:     28 bajtów                            |
[00:18:36] +-----+
[00:18:36]
[00:18:36] >> [DYREKTOR] Magazyn ZAMKNIĘTY - blokada operacji!
...
[00:18:36] [DOSTAWCA-D] Magazyn zamknięty - czekam...
[00:18:36] [DOSTAWCA-A] Magazyn zamknięty - czekam...
[00:18:36] [PRACOWNIK-2] *** WYPRODUKOWANO CZEKOLADE - TYP_2 (A+B+D) #11 ***
[00:18:36] [PRACOWNIK-2] Magazyn zamknięty - czekam...
[00:18:36] [DOSTAWCA-B] Magazyn zamknięty - czekam...
[00:18:36] [DOSTAWCA-C] Magazyn zamknięty - czekam...
[00:18:38] [DOSTAWCA-D] Magazyn zamknięty - czekam...
[00:18:38] [DOSTAWCA-A] Magazyn zamknięty - czekam...
[00:18:38] [PRACOWNIK-2] Magazyn zamknięty - czekam...
[00:18:38] [DOSTAWCA-B] Magazyn zamknięty - czekam...
[00:18:38] [DOSTAWCA-C] Magazyn zamknięty - czekam...
[00:18:39] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #11 ***
[00:18:39] [PRACOWNIK-1] Magazyn zamknięty - czekam...
[00:18:40] [DOSTAWCA-D] Magazyn zamknięty - czekam...
```

Ponowne wysłanie przez dyrektora polecenie "2" - otwarcie magazynu

```

2
[00:18:50]
[00:18:50] +-----+
[00:18:50] |                STAN MAGAZYNU                |
[00:18:50] +-----+
[00:18:50] | Skladnik A:   0 szt. (  0 bajtow)                |
[00:18:50] | Skladnik B:   0 szt. (  0 bajtow)                |
[00:18:50] | Skladnik C:   9 szt. ( 18 bajtow)                |
[00:18:50] | Skladnik D:   8 szt. ( 24 bajtow)                |
[00:18:50] +-----+
[00:18:50] | Zajete:    42 /   70 bajtow ( 60%)                |
[00:18:50] | Wolne:     28 bajtow                               |
[00:18:50] +-----+
[00:18:50]
[00:18:50] >> [DYREKTOR] Magazyn OTWARTY - wznowiono operacje!
...
[00:18:51] [PRACOWNIK-1] Czekam na komplet skladnikow...
[00:18:52] [PRACOWNIK-2] Czekam na komplet skladnikow...
[00:18:52] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 43/70 |
[00:18:52] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 44/70 |
[00:18:52] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 46/70 |
[00:18:52] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 49/70 |
[00:18:52] [PRACOWNIK-1] Pobrano: A, B, C | Magazyn zajety: 45/70 |
[00:18:53] [DOSTAWCA-C] Dostarczono 1 x C | Magazyn zajety: 47/70 |
[00:18:54] [DOSTAWCA-A] Dostarczono 1 x A | Magazyn zajety: 48/70 |
[00:18:54] [DOSTAWCA-B] Dostarczono 1 x B | Magazyn zajety: 49/70 |
[00:18:54] [PRACOWNIK-2] Pobrano: A, B, D | Magazyn zajety: 44/70 |
[00:18:54] [DOSTAWCA-D] Dostarczono 1 x D | Magazyn zajety: 47/70 |
[00:18:55] [PRACOWNIK-1] *** WYPRODUKOWANO CZEKOLADE - TYP_1 (A+B+C) #12 ***

```

Rezultat testu jest zgodny z oczekiwaniami:

- Pracownicy i dostawcy odbierają sygnał od dyrektora i wstrzymują swoją pracę. Stan magazynu jest taki sam w trakcie zamknięcia jak i w momencie ponownego otwarcia. Test zaliczony.

8. Linki do Istotnych Fragmentów Kodu

a. Tworzenie i obsługa plików

Zapis do pliku raportu (write, open, close):

- fopen() - [utlis.c#L342](#)
- fwrite() - [utlis.c#L348](#)
- fclose() - [utlis.c#L349](#)
- fread() - [utlis.c#L369](#)

b. Tworzenie procesów

fork() - Uruchamianie dostawców:

- fork() - [dyrektor.c#L153](#)

execl() - Uruchomienie programu potomnego:

- execl() - [dyrektor.c#L155](#)

wait() / waitpid() - Zbieranie procesów:

- waitpid() - [dyrektor.c#L285](#)

exit() - Zakończenie procesu potomnego:

- exit() - [dyrektor.c#L157](#)

c. Tworzenie i obsługa wątków

Nie wykorzystano w projekcie

d. Obsługa sygnałów

- `sigaction()` - [dyrektor.c#L78](#)

`signal()` - Rejestracja handlera w procesach potomnych:

- `signal()` - [dostawca.c#L22](#)

`kill()` - Wysyłanie sygnałów do procesów:

- `kill()` - [dyrektor.c#L33](#)

e. Synchronizacja procesów (semafony)

`semget()` - Utworzenie zestawu semaforów:

- `semget()` [utils.c#L1997](#) -

`semctl()` - Inicjalizacja wartości semaforów:

- `semctl()` [utils.c#L215](#)

`semop()` - Operacje na semaforach (wait/signal):

- `semop()` [dostawca.c#L106](#)

`sem_wait()` i `sem_signal()` - Implementacja pomocniczych funkcji dla operacji semaforowych:

- Implementacja `sem_wait()` - [utils.c#L289-L302](#) -
- Implementacja `sem_signal()` [utils.c#L304-L314](#) -

f. Łączy nazwane i nienazwane

Nie wykorzystano w projekcie

g. Segmenty pamięci dzielonej

`shmget()` - Utworzenie segmentu pamięci dzielonej:

- `shmget()` - [utils.c#L153](#)

`shmat()` - Dołączenie do pamięci dzielonej:

- `shmat()` [utils.c#L165](#)

`shmdt()` - Odłączenie od pamięci dzielonej:

- `shmdt()` - [utils.c#L178](#)

`shmctl()` - Usunięcie segmentu pamięci:

- `shmctl()` - [utils.c#L187](#)

h. Kolejki komunikatów

Nie wykorzystano w projekcie

i. Gniazda

Nie wykorzystano w projekcie

9. Podsumowanie

Projekt **Fabryka Czekolady** w pełni realizuje założenia wykorzystania mechanizmów IPC w systemach Unix/Linux.

System jest **odporny na zakleszczenia**, **wolny od procesów zombie**, obsługuje **graceful shutdown** i zapewnia **persystencję danych**. Implementacja ring bufferów FIFO oraz atomowych operacji semaforowych gwarantuje poprawną synchronizację w środowisku wieloprotocowym.
